Windows resides in */EFI/Microsoft/Boot/bootmgfw.efi*.

By now, you must have realised something -- UEFI allows binaries to be stored as files with an *.efi* extension in the ESP. In fact, EFI binaries are nothing but 64-bit DLL files (Windows-style shared libraries) that do not link to any other shared objects, do not contain a symbol table, and specify their own application type (0x10 for UEFI applications, 0x11 and 0x12 for drivers). UEFI applications have access to quite a lot of the hardware in the computer and all of the memory. UEFI uses a flat memory model where the entire address space is identity-mapped (i.e., the virtual address and physical address of a location in memory is the same).

This brings me to the final bit of information about UEFI that you need to know before you can start playing with it—UEFI also has a shell, which can be used pretty much like a DOS prompt. It allows the running of system maintenance tasks and executes other EFI programs, which could be bootloaders for other operating systems. Your UEFI implementation may ship with an EFI shell; then again, it might not. If it doesn't, you can always download a copy and throw it into your ESP.

So that's UEFI for end users—a system set-up utility and a glorified bootloader, all rolled into one.

No GRUB?

Loading Linux with UEFI is easier than using BIOS. All other factors remaining constant, UEFI eliminates the need for GRUB.

First, some theory—you don't need a bootloader to boot Linux using UEFI. The UEFI environment can execute arbitrary executables that conform to the UEFI executable format, and the Linux kernel binary can be built as one such executable. This is known as the EFISTUB method, since it attaches a small stub executable to the kernel that effectively makes it an EFI binary. EFISTUB kernels can be loaded by conventional bootloaders without any ill-effects, so most distributions nowadays ship with EFISTUB enabled. You can check if your kernel is EFISTUB-enabled by running the following command:

```
$: zcat /proc/config.gz | grep EFI
```

What you're looking for are the following four lines:

```
CONFIG_EFI_PARTITION=y
CONFIG_EFI=y
CONFIG_EFI_STUB=y
CONFIG_FB_EFI=y
```

If these four configuration elements are enabled, then your kernel is EFISTUB enabled. If not, you'll need to recompile your kernel with these four options enabled, plus the next one:

```
CONFIG_EFI_VARS=m
```

This option can also be set to *y* instead of *m*, which simply means that the *efivars* module will be built into the kernel. You'll need this to manipulate your UEFI NVRAM and create bootloader entries in your firmware.

Migrating from BIOS to UEFI

If you're currently using a BIOS-based booting system and want to migrate to UEFI, this section is for you. It's very easy to switch from BIOS to UEFI, and it can be done mostly without any data loss. All you need to do is to reformat your boot partition using FAT32 (if you aren't using a separate boot partition, you should).

If you're using a separate boot partition, the migration is quite easy. If you're not using a boot partition, the procedure is a bit more involved. In the latter case, please make sure you have some unallocated space in your hard drive because a boot partition has to be created.

Note: The following procedure, if done wrongly, may result in an unbootable system and/or partial or complete data loss. Reader discretion is advised. I'm not responsible if you lose data.

Begin by installing the *gptfdisk* package into your system, and run the following command:

```
$: sudo gdisk /dev/sdX
```

…replacing *sdX* with your drive.

Simply launching *gdisk* creates a UEFI partition table equivalent to your current MBR layout in memory. To commit the GPT to disk, type *w* and press 'Enter'. To back out, type *q* and press 'Enter'.

Now that the drive has a GPT instead of an MBR, you can't boot using your BIOS any more. Do not reboot your computer. At this point, it would be a wise idea to check out what your disk looks like.

Run the following command:

```
$: sudo gdisk -l /dev/sdX
```

You should get a listing of your partition table. Mine looks like this:

```
GPT fdisk (gdisk) version 0.8.7

Partition table scan:
  MBR: protective
  BSD: not present
  APM: not present
  GPT: present

Found valid GPT with protective MBR; using GPT.
Disk /dev/sda: 976773168 sectors, 465.8 GiB
```